Ultrasonic NDE Wave-Propagation Exercise

Project goal

The project estimates bulk ultrasonic wave speeds in isotropic solids and examines how those speeds respond to stiffness and density changes. It also provides a cautious, reproducible way to collect sample material properties from permitted public web pages.

Engineering context

Ultrasonic non-destructive evaluation (NDE) uses elastic waves to inspect components without cutting or damaging them. Longitudinal waves involve particle motion parallel to propagation; transverse waves involve particle motion perpendicular to propagation. Their speeds depend on elastic stiffness and mass density.

Main deliverables

The compute workflow produces a validated results CSV, a printed Markdown table, and two PNG sensitivity plots. The scraper produces a Task-1-compatible input CSV and a separate failed-URL log.

1. Project files and responsibilities

compute_wave_velocities.py

Reads the material parameter CSV, validates required columns and physical constraints, converts Young's modulus from GPa to Pa, computes Lamé constants and wave speeds, writes results, and creates the stiffness and density plots.

scrape_material_parameters.py

Visits only URLs explicitly placed in CONFIG['source_urls']. It checks robots.txt, verifies a configured terms page, sends a clear User-Agent, waits at least two seconds between requests, parses best-effort property values, and records failures without crashing.

material_parameters.csv

The input schema is name, E_GPa, nu, rho_kg_m3, source_url. The current example contains Aluminum 6061-T6 and carbon steel. The example source links and values must be independently verified.

README.md

Documents installation, commands, equations, configuration values, units, and assumptions that should be checked before engineering use.

2. Inputs, units, and reproducibility

Configuration

The compute script keeps file paths, baseline material, number of sensitivity points, and scale-factor limits in CONFIG. The current baseline is Aluminum 6061-T6; the sensitivity range is 0.5 to 1.5 with 50 evenly spaced points.

Input meanings

E_GPa is Young's modulus in gigapascals. nu is dimensionless Poisson's ratio. rho_kg_m3 is density in kilograms per cubic metre. source_url records provenance and is not used in the equations.

Unit conversion

Because the equations are SI-based, E_GPa is multiplied by 10^9 before use. The density is already in kg/m^3, so no density conversion is performed.

Required dependencies

NumPy supplies numerical operations, pandas supplies CSV and table handling, Matplotlib supplies plots and PDF generation, requests performs HTTP requests, BeautifulSoup parses HTML, and tabulate supports pandas Markdown output.

3. Elastic-wave equations

Material constants

For an isotropic, homogeneous, linearly elastic solid, the shear modulus is mu = E / [2(1 + nu)]. The first Lamé constant is lambda = E nu / [(1 + nu)(1 - 2 nu)].

Wave speeds

The longitudinal speed is v_L = sqrt[(lambda + 2 mu) / rho]. The transverse speed is v_T = sqrt[mu / rho]. The script also reports v_L/v_T as a useful comparison between the modes.

Physical interpretation

Increasing stiffness increases the restoring force and therefore increases both speeds. Increasing density increases inertia and therefore decreases both speeds when stiffness is held fixed.

Why Poisson's ratio matters

Poisson's ratio affects both Lamé constants and especially the longitudinal response. The validation restricts it to 0 < nu < 0.5, the stable range used by this isotropic-solid formulation.

4. Computation workflow: step by step

Step 1 - Read data

main() loads material_parameters.csv with pandas. The calculation expects the five required columns and preserves material names.

Step 2 - Normalize numeric fields

The calculation copies the input and converts E_GPa, nu, and rho_kg_m3 with pandas.to_numeric. Invalid non-numeric text and missing values produce a clear row-specific ValueError.

Step 3 - Validate physics

Each row must have E > 0, rho > 0, and 0 < nu < 0.5. Missing columns also raise an explicit error before any square root is taken.

Step 4 - Calculate

The script converts E to Pa, computes mu and lambda, then evaluates v_L, v_T, and v_L/v_T using vectorized pandas and NumPy operations.

Step 5 - Save and display

Results are written to wave_velocity_results.csv and printed as a Markdown table. The output columns are name, E, nu, rho, mu, lambda, v_L (m/s), v_T (m/s), and v_L/v_T.

Computed bulk-wave results

name	E	nu	rho	mu	lambda	v_L (m/s)	v_T (m/s)	v_L/v_T
Aluminum 6061-	6.89e+10	0.330	2700	2.59e+10	5.028e+10	6148.9	3097.3	1.9852
Carbon steel	2e+11	0.300	7850	7.692e+10	1.154e+11	5856.4	3130.4	1.8708

E, mu, and lambda are shown in Pa; rho is kg/m^3; velocities are m/s.

5. Sensitivity analysis and prediction checks

Baseline selection

The script finds the configured baseline row by exact material name. If it is missing, execution stops with an explicit error rather than silently choosing another material.

Stiffness sweep

The baseline Young's modulus is multiplied by 50 evenly spaced scale factors from 0.5 to 1.5. Poisson's ratio and density remain fixed. The code predicts that both wave speeds increase monotonically.

Density sweep

The baseline density is multiplied by the same scale-factor range. Young's modulus and Poisson's ratio remain fixed. The code predicts that both wave speeds decrease monotonically.

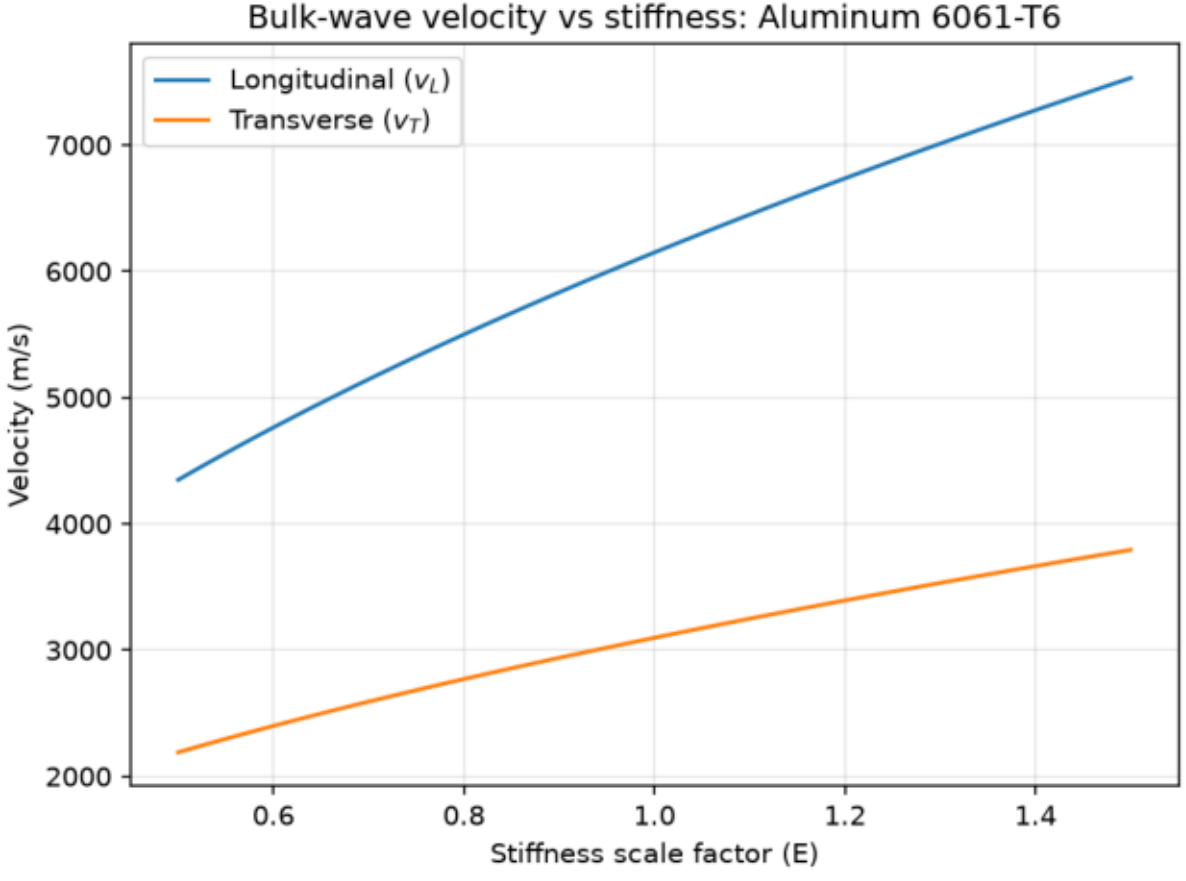
Numerical verification

After each sweep, NumPy.diff checks every adjacent pair. The script prints whether all stiffness differences are positive and all density differences are negative, directly comparing computed behavior with the comments written before computation.

Plot outputs

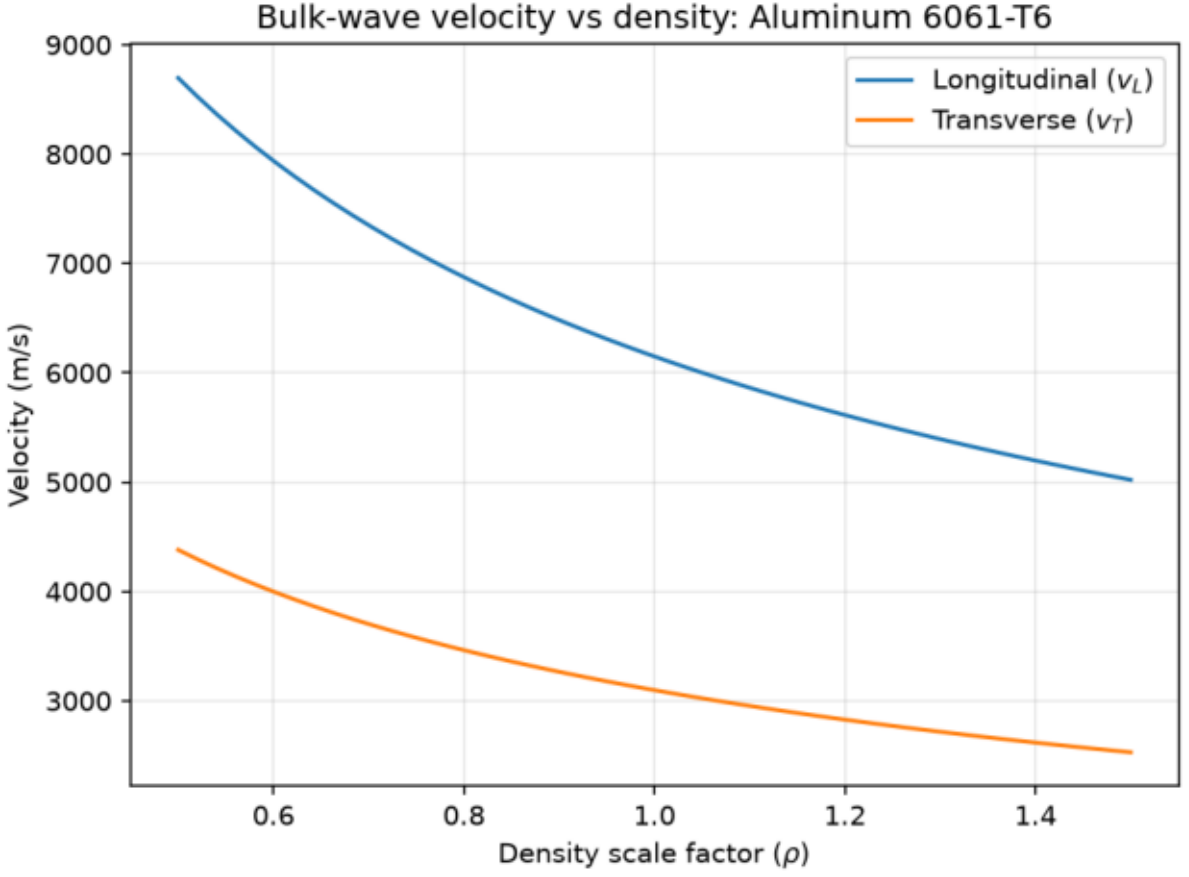
velocity_vs_stiffness.png and velocity_vs_density.png each show longitudinal and transverse speeds, labeled axes in physical units, a title, legend, grid, and a 150 dpi export.

Sensitivity result: stiffness



Expected result: both curves rise as the stiffness scale factor increases. The transverse curve remains below the longitudinal curve for this baseline.

Sensitivity result: density



Expected result: both curves fall as the density scale factor increases, because velocity is proportional to the inverse square root of density when the elastic constants are held fixed.

6. Scraping workflow and responsible-use controls

URL configuration

The scraper starts with an empty source_urls list. A user must add specific public pages only after reviewing the site's current terms and confirming that automated access is permitted.

Robots and terms

For each target, urllib.robotparser reads the site's robots.txt and checks the configured User-Agent. The script also fetches a configured terms URL for supported domains. Unknown domains are rejected until a terms URL is deliberately configured.

Request pacing

polite_get enforces a minimum two-second interval between all HTTP requests, including terms-page and data-page requests. A 20-second timeout limits how long one request can block the run.

Parsing and missing data

BeautifulSoup turns page content into text. Regular expressions look for Young's modulus in GPa, Poisson's ratio, and density in kg/m^3. Unrecognized fields become None, while failed URLs and their errors are written to scrape_failed_urls.txt.

Important limitation

The parser is intentionally generic and best-effort. It may not understand a site's table layout, alternate unit notation, dynamic content, or a property label that differs from the patterns. Every scraped row must be manually reviewed before computation.

7. Assumptions, verification, and final run sequence

Assumptions to verify

Confirm that all values use the stated units; that the material is homogeneous, isotropic, and linearly elastic; that E, nu, and rho refer to the same alloy condition and temperature; and that scraped records are from trustworthy sources.

Model limitations

The bulk formulas do not include attenuation, dispersion, texture, anisotropy, porosity, temperature dependence, frequency dependence, or boundary effects. They are a baseline elastic-wave calculation, not a complete ultrasonic inspection model.

Recommended run sequence

Install dependencies, review and edit the CSV, run the compute script, inspect validation messages and plots, and compare the numerical results with trusted reference data. For scraping, review terms and robots first, add permitted URLs, run the scraper, inspect the failed URL log and missing values, then run the compute script on the validated scraped CSV.

Reproducibility record

The current configuration uses Python 3.10 or newer, Aluminum 6061-T6 as baseline, 50 points, and scale factors 0.5 through 1.5. The generated CSV and PNG files provide an auditable record of the calculation associated with this report.